SYSTEM DESIGN & ARCHITECTURE | SIMPLE TERMS GUIDE | MULTI-AGENT SECURITY | ZERO-DEPENDENCY

AEGIS-AGENT System Design & Architecture

How this autonomous cybersecurity & self-healing platform works — explained in simple, intuitive, step-by-step concepts for engineers, leaders, and evaluators.

DESIGN PARADIGM
Autonomous Multi-Agent SecOps

CORE PHILOSOPHY
Closed-Loop: Discover → Prove → Fix → Verify

RUNTIME ARCHITECTURE
Pure Node.js + Persistent SQLite

SAFETY MODEL
Non-Destructive Sandbox SEC-LOCAL-8080

1. The Big Picture: How It Works in Plain English

Imagine you own a large commercial building with dozens of doors, security cameras, and locks.

How Traditional Security Scanners Work:

Traditional scanners (like old-school SAST and DAST) act like an overly sensitive smoke alarm. They walk down the hallway, smell someone making toast, and **sound 500 panic alarms a day**.

They don't tell you if there is an actual fire, they don't test if the door is really unlocked, and they **never repair the lock**. Your security team spends all day reading false alarms and typing reports.

How AEGIS-AGENT Works:

AEGIS-AGENT acts like a **professional, automated security SWAT & locksmith crew** that works together in a synchronized team of 5 specialized agents:

They read the architectural blueprint first, gently jiggle the lock to see if it clicks open, take a timestamped photo of the flaw, craft a brand new custom replacement lock, test it twice to make sure it works, and ask for your permission before bolting it in!

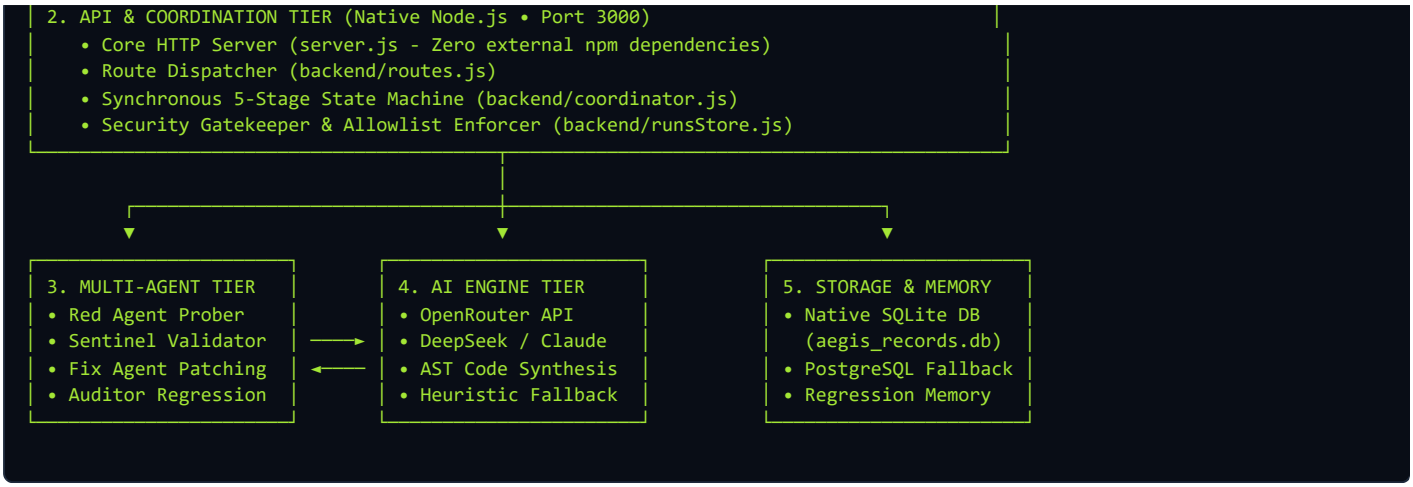
🎭 **The 5 Agents Explained as Real-World Crew Roles:**

- **01. The Blueprint Reader (Target Research):** Inspects the Git repository and API specification to map all entrances before anyone touches a door.
- **02. The Boundary Guard (Scope Controller):** Ensures tests ONLY run on the authorized practice building, never touching public neighbors or the real world.
- **03. The Safe Lockpicker (Red Agent):** Tries a non-destructive test key (e.g. `' OR '1'='1`) to see if the door latch clicks open.
- **04. The Detective (Sentinel Agent):** Inspects the lock mechanism to confirm: *"Yes, the latch gave way—this is 100% real, not a false alarm."*
- **05. The Master Locksmith (Fix Agent / AI):** Uses AI to forge a custom-fitted, secure replacement bolt (writes a parameterized code patch).
- **06. The Certified Inspector (Auditor Agent):** Tests the new bolt: *Can the lockpicker break it? NO. Can the keyholder open it? YES.*
- **07. The Building Owner (Human Operator):** Reviews the photo evidence and lock receipt, clicks **Approve**, and merges the fix.

2. High-Level System Architecture (Tiers & Components)

Under the hood, AEGIS is designed as a lightweight, high-performance system with **zero external package dependencies**. Here is how the system components connect:




2. API & COORDINATION TIER (Native Node.js • Port 3000)
• Core HTTP Server (server.js - Zero external npm dependencies)
• Route Dispatcher (backend/routes.js)
• Synchronous 5-Stage State Machine (backend/coordinator.js)
• Security Gatekeeper & Allowlist Enforcer (backend/runsStore.js)
3. MULTI-AGENT TIER
• Red Agent Prober
• Sentinel Validator
• Fix Agent Patching
• Auditor Regression
4. AI ENGINE TIER
• OpenRouter API
• DeepSeek / Claude
• AST Code Synthesis
• Heuristic Fallback
5. STORAGE & MEMORY
• Native SQLite DB
(aegis_records.db)
• PostgreSQL Fallback
• Regression Memory

3. The Lifecycle of a Security Run (Step-by-Step Data Flow)

Here is exactly what happens behind the scenes from the second you click a button until the final patch is delivered:

STAGE / STEP	WHAT YOU DO IN THE UI	WHAT THE SYSTEM DOES BEHIND THE SCENES	SIMPLE ANALOGY / RESULT
Step 1: Target Ingestion	Select a preset chip or enter a staging URL on the **Target Research** page.	The system sends a request to `/api/research/ingest`, parses the OpenAPI spec, scans repository route declarations, and extracts parameters.	Reads the building blueprint to find all doors and windows.
Step 2: Scope Verification	Check the authorization checkbox and click **Launch Live War Room**.	`runsStore.validateNewRunRequest()` checks target allowlist. If external URL, maps it to safe sandbox `SEC-LOCAL-8080`. Creates run record.	The guard checks your permit to make sure you only test your own building.
Step 3: Red Probing	War Room stopwatch starts (`00:01.2s`). Red Agent node lights up green.	Red Agent dispatches a safe syntax-breaker string (like `admin' OR '1'='1`). The target database encounters a syntax error and returns a 500 trace.	Gently jiggles the lock to see if the latch trips open.
Step 4: Sentinel Evidence	Sentinel Agent node turns green; progress reaches 50%.	Sentinel Agent evaluates the HTTP response body and stack trace. It confirms the database error contains raw query failure proof, ruling out false positives.	Takes an evidentiary photo: "The latch is definitely broken."
Step 5: AI Fix Synthesis	Fix Agent node activates; progress reaches 75%.	Fix Agent takes the vulnerable file snippet (e.g. `authService.js:42`) and asks OpenRouter AI to write a minimal parameterized query unified diff.	The master locksmith crafts a custom-fitted, secure replacement bolt.
Step 6: Auditor Regression	Auditor Agent node turns green; progress reaches 100% (`COMPLETED`).	Auditor Agent spins up an isolated worktree. It re-tests the exploit (blocked with `400`) and legitimate user logins (succeeds with `200`).	Tests the new lock: burglar is locked out, but legitimate family keys work!
Step 7: Operator Approval	Review the diff and click **Approve Patch** or **Export Report**.	The system marks the finding as `APPROVED_FOR_MERGE` and logs the verification fingerprint to persistent SQLite memory.	Building owner signs the delivery receipt.

4. How the Agents Work Under the Hood (In Simple Terms)

1. Red Agent (The Ethical Prober)

Question: *Does it do dangerous hacking?*
Answer: NO. The Red Agent is strictly bounded to non-destructive, safe syntax breaker probes.

Instead of dumping database tables or stealing files, it delivers controlled input sequences (e.g. single quotation marks or path traversals) designed to test input validation boundaries and capture the server's error response.

2. Sentinel Agent (The False-Positive Filter)

Question: *How does it know if a vulnerability is real?*
Answer: By requiring **empirical proof**.

Traditional scanners guess a vulnerability exists by checking library versions. Sentinel requires proof: it examines the captured HTTP status, database stack trace, and response body. If the evidence does not clearly prove exploitability, it halts the workflow with `NEEDS_REVIEW`.

3. Fix Agent (The AI Patch Maker)

Question: *Does AI rewrite the whole app?*
Answer: NO. It writes a surgical, minimal diff.

The Fix Agent isolates only the 3–5 lines of code responsible for the vulnerability (e.g. replacing string concatenation

4. Auditor Agent (The Independent Tester)

Question: *What if the AI patch breaks something?*
Answer: That is why the Auditor Agent exists!

The Auditor runs a **dual-verification suite** in a temporary disposable directory:
• *Security Check:* Does the hack still work? (Must fail)

with SQL parameter binding `[?]`). It produces a standard Git unified diff that any developer can review in 10 seconds.

• *Functional Check:* Does normal login still work? (Must pass)

5. Storage & Database Architecture (Explained Simply)

AEGIS is built to be **zero-maintenance and completely self-contained**. You don't need to install Docker, configure Postgres, or manage external database servers just to run a test:

Embedded Native SQLite (`node:sqlite`)

When you run `node server.js`, the server automatically initializes a high-performance local SQLite database file at `aegis_records.sqlite`.

- Stores every workflow run, timestamp, and duration.
- Stores captured evidence, HTTP traces, and call stacks.
- Stores proposed patches, approvals, and regression results.
- **Zero setup required:** works instantly on any computer.

Enterprise PostgreSQL Integration (Optional)

If you provide a `DATABASE_URL` in your `.env` file (e.g. connecting to Supabase or AWS RDS), AEGIS automatically connects via a pooled PostgreSQL client.

- Applies versioned migrations from `migrations/001_initial_schema.sql`.
- Allows multiple operators to view runs across teams.
- If PostgreSQL connection ever fails, it automatically falls back to native SQLite without crashing!

6. Security Guardrails: Why AEGIS is 100% Safe

Because AEGIS is an autonomous security tool, it was built with **ironclad defense-in-depth safety boundaries**:

POTENTIAL RISK	HOW TRADITIONAL AI TOOLS HANDLE IT	HOW AEGIS GUARANTEES SAFETY
Scanning Wrong Targets	AI agents can follow arbitrary URLs and accidentally attack public infrastructure.	**Strict Allowlist:** Only explicitly authorized local training application endpoints are permitted. External URLs are mapped into safe local simulation containers.
Accidental Data Loss	Destructive payloads can delete rows, drop tables, or crash production containers.	**Syntax-Breaker Only:** Probes only test parsing boundaries to capture error traces; zero data deletion or modification commands exist in the payload catalog.
Unwanted Code Commits	Autonomous tools push code directly to GitHub branches without human review.	**Mandatory Human Gate:** Candidate patches are synthesized in disposable worktrees and require explicit human operator approval before merging.
Credential Leaks	API keys and tokens get accidentally committed to public Git repositories.	**Push Protection & .env:** All secrets are strictly kept in `.env` and excluded in `.gitignore`. The web server blocks direct HTTP requests to hidden files.

7. Evaluator & Judge Cheatsheet: 30-Second Summary

🎯 What Makes AEGIS-AGENT Stand Out in 3 Bullet Points:

1. **It Doesn't Just Detect — It Proves and Heals:** Instead of dumping a list of 50 speculative warnings on a developer's desk, AEGIS captures real proof, generates a working code patch, and verifies that the patch works in under 60 seconds.
2. **Zero Hallucinations in Execution:** Even though an LLM writes the code patch, an independent Auditor Agent physically executes a regression test suite against the patch before any human is asked to approve it.
3. **Pure Engineering Elegance:** Built with zero external dependencies, native Node.js core modules, high-performance embedded SQLite, pure Vanilla JavaScript, and an XBOW-inspired visual war room with Web Audio sound telemetry.